



Day 1. Python 기초와 개발 환경

C/C++ · Arduino · AVR 경험자를 위한 Python 입문





오늘의 학습 목표

- 1 Python의 특징 이해 
- 2 C/C++와의 차이 파악 
- 3 개발 환경 구성 
- 4 변수·자료형·연산 익히기 
- 5 간단한 입출력 프로그램 작성 

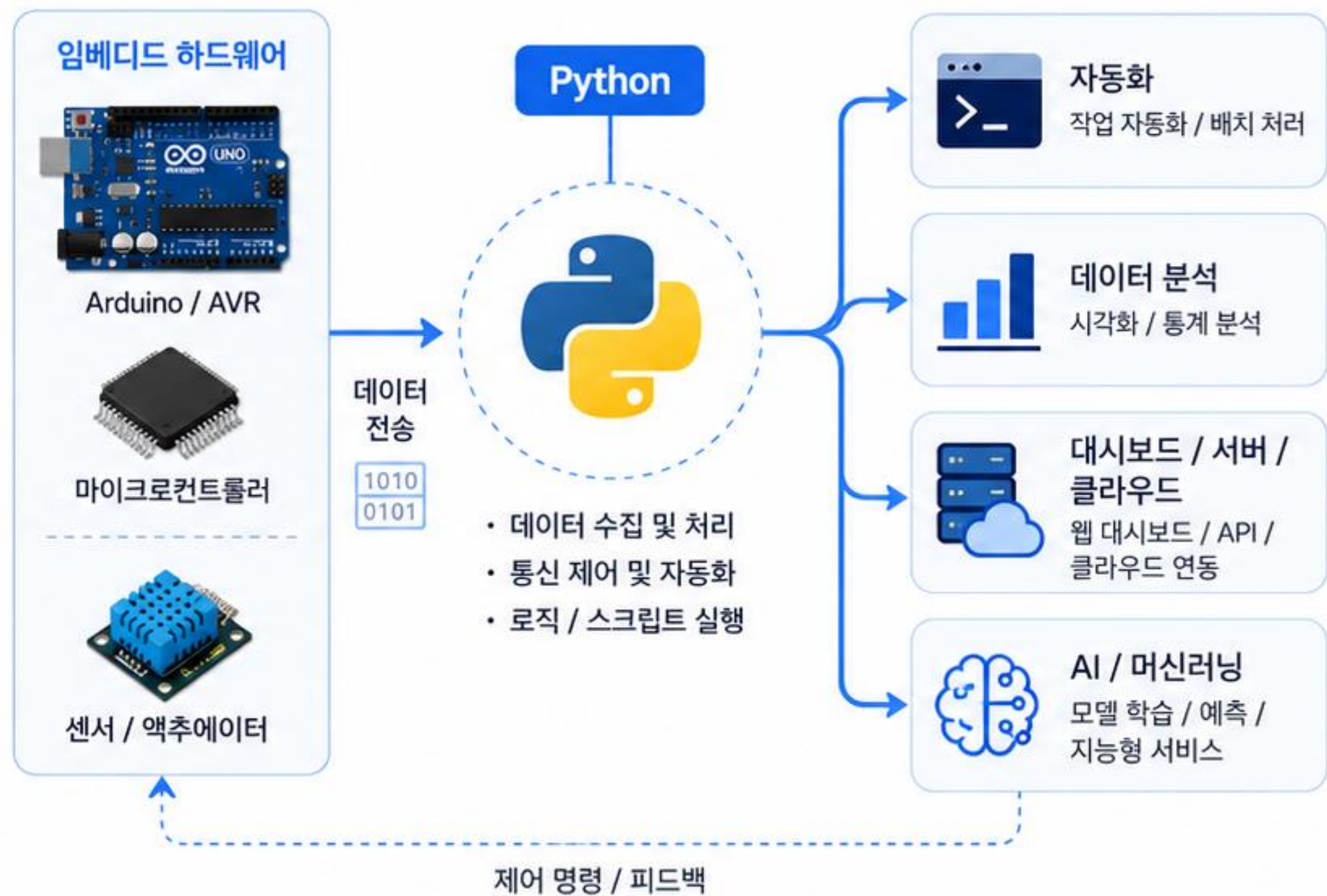
Day 1 수업에서 무엇을 배우는지 한눈에 정리합니다.



왜 임베디드 개발자에게 Python이 필요한가?

- 1 빠른 프로토타이핑
- 2 센서 데이터 처리
- 3 시리얼 통신 자동화
- 4 테스트 스크립트 작성
- 5 서버·클라우드 연동
- 6 AI·데이터 분석 확장

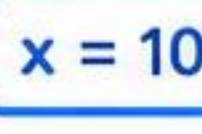
C/C++로 제어하고, Python으로 연결하고 자동화합니다.





Python이란?

쉽고 강력한 범용 프로그래밍 언어

- 1**  **인터프리터 언어**
코드를 한 줄씩 실행하며 빠르게 테스트 가능
- 2**  **문법이 간결함**
가독성이 높고 배우기 쉬움
- 3**  **동적 타이핑**
변수 타입을 미리 선언하지 않음
- 4**  **객체지향 지원**
클래스와 객체 기반 설계 가능
- 5**  **라이브러리가 풍부함**
웹, 데이터, AI, 자동화, IoT까지 확장 가능

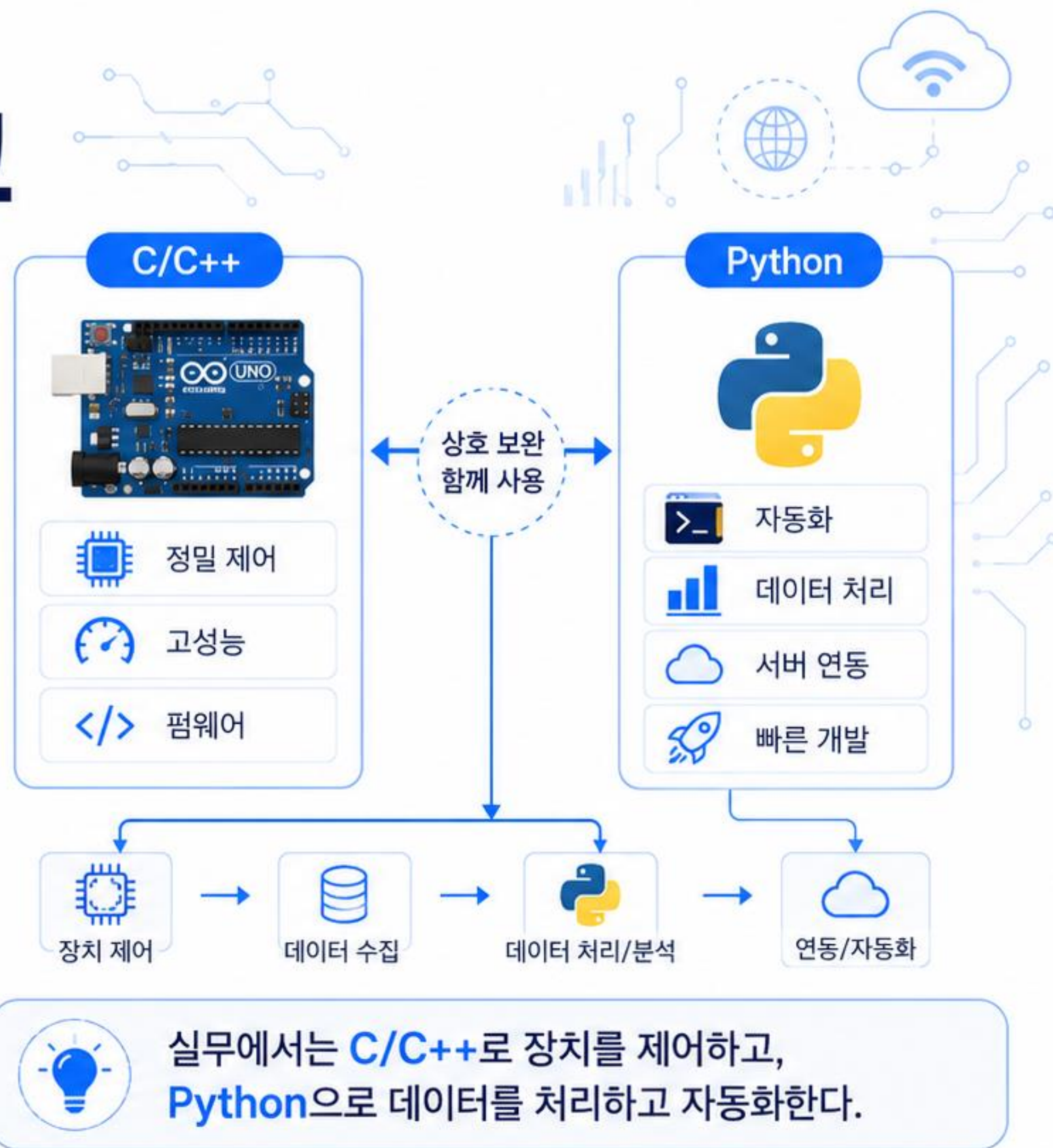
Python은 빠른 개발, 자동화, 데이터 처리에 강한 언어입니다.



C/C++와 Python 비교

임베디드 제어는 C/C++, 빠른 개발과 자동화는 Python

	C/C++	Python
1 컴파일/실행 방식	컴파일 후 실행	인터프리터로 바로 실행
2 타입 선언	변수 타입을 미리 선언	값에 따라 타입이 결정됨
3 문법 복잡도	상대적으로 복잡함	간결하고 읽기 쉬움
4 메모리 관리	직접 관리 가능	자동 메모리 관리
5 개발 속도	구현에 시간 소요	빠른 프로토타이핑 가능
6 하드웨어 제어	매우 강함	직접 제어보다 연동에 강함
7 자동화/데이터 처리	상대적으로 불편	매우 강함



Python 개발 환경 구성

Python을 사용하기 위한 기본 환경을 준비해 봅시다.

-  **Python 설치**
공식 홈페이지(<https://www.python.org>)에서
최신 버전 설치 (Windows 기준)
-  **VS Code 설치**
Visual Studio Code 설치 후 실행
(<https://code.visualstudio.com>)
-  **확장팩(Python Extension) 설치**
VS Code 좌측 확장 아이콘 → "Python" 검색
→ Microsoft의 Python 확장 설치
-  **터미널 실행 및 확인**
VS Code 하단 터미널 열기 (Ctrl + ~)
python --version 명령으로 설치 확인
-  **실행 테스트**
test.py 파일을 만들고 간단한 코드 실행하여
Python이 정상 동작하는지 확인

환경 구성 흐름도



💡 설치 확인 체크리스트

- ✓ python --version 실행 시 버전 정보가 출력된다.
- ✓ VS Code에서 .py 파일이 실행된다.
- ✓ 터미널에서 Python 명령어가 정상 동작한다.

TIP

- Python은 3.8 이상 버전을 권장합니다.
- 작업 폴더는 영문 경로를 사용하는 것이 좋습니다.
- 문제가 발생하면 터미널 오류 메시지를 확인하고 검색해 보세요!

첫 번째 Python 실행

인터프리터와 스크립트 방식으로 Python을 처음 실행해 봅시다.

1



터미널 열기

명령 프롬프트 또는 VS Code 터미널 실행

2



Python 인터프리터 실행

터미널에서 python 입력 후 Enter

3



첫 출력 해보기

print("Hello, Python") 입력

4



스크립트 파일 작성

hello.py 파일을 만들고 코드 저장

5



실행 결과 확인

python hello.py 명령으로 실행

인터프리터 실행

```

>>> print("Hello, Python")
Hello, Python
>>> 
    
```

hello.py 실행

```

1 print("Hello, Python")

$ python hello.py
Hello, Python
    
```



터미널



코드 작성



실행

핵심 포인트

- ✓ 인터프리터는 바로 실행하며 테스트에 유리
- ✓ 스크립트 파일은 반복 실행과 저장에 유리
- ✓ 에러가 나면 메시지를 읽고 수정

TIP

- 설치 확인: python --version
- 파일 저장 후 실행 습관 들이기
- VS Code와 터미널을 함께 사용하면 편리

인터프리터 방식 vs 스크립트 방식

Python 코드를 실행하는 두 가지 기본 방법을 비교합니다.

인터프리터 방식

>_ 터미널

```
>>> x = 10
>>> print(x * 2)
20
>>> 
```



한 줄씩 바로 실행

입력 즉시 실행되어 빠르게 결과 확인



빠른 테스트에 적합

간단한 코드 테스트와 실험에 유리



결과를 즉시 확인 가능

출력 결과를 바로 확인할 수 있음

스크립트 방식

hello.py

```
1 def main():
2     print("Hello, Python")
3
4 if __name__ == "__main__":
5     main()
```

>_ 터미널

```
$ python hello.py
Hello, Python
```



파일로 저장하여 관리

코드를 파일로 저장해 체계적으로 관리



반복 실행과 수정이 편리

수정 후 다시 실행하는 과정이 용이



프로그램 작성에 적합

여러 기능을 포함한 프로그램 개발에 적합



입력

코드 한 줄 입력



즉시 실행

바로 실행됨



결과 확인

결과 즉시 출력



파일 작성

코드를 파일에 작성



저장

파일로 저장



실행

명령으로 실행



핵심 정리



빠른 테스트는 인터프리터

간단한 테스트나 실험은 인터프리터 방식이 빠르고 효율적입니다.



프로그램 작성은 스크립트

여러 기능을 가진 프로그램은 스크립트 방식으로 작성하는 것이 적합합니다.



실무에서는 두 방식을 함께 사용합니다.

상황에 맞게 두 방식을 적절히 선택하고 함께 사용하는 것이 중요합니다.

09 Python 문법의 핵심 규칙

Python은 간결하지만, 몇 가지 규칙을 반드시 지켜야 합니다.



Point

Python은 '들여쓰기'로 코드 블록을 구분합니다.
정확한 문법 규칙을 지키는 것이 중요합니다!



1 들여쓰기 (Indentation)

- 코드 블록은 들여쓰기로 구분합니다.
- 보통 4칸(스페이스) 또는 Tab을 사용합니다.



2 대소문자 구분 (Case Sensitive)

- Python은 대소문자를 구분합니다.
- 변수명, 함수명, 키워드에 주의하세요.



3 주석 사용 (Comments)

- 한 줄 주석: #
- 여러 줄 주석: ''' 또는 '''



4 콜론 (:) 사용

- if, for, while, def, class 등 블록을 시작할 때 콜론(:)을 사용합니다.



5 코드 블록 구조

- 콜론(:) 뒤에 들여쓰기로 블록을 구성합니다.
- 들여쓰기가 끝나면 블록도 종료됩니다.

✓ 올바른 예

```
if x > 0:
    print("양수")
print("끝")
```

→ 들여쓰기로
블록을 구분

```
count = 10
Count = 20
print(count)
```

→ 대소문자가
다른 변수

```
# 한 줄 주석입니다
print("Hello")
"""
여러 줄 주석
입니다.
"""
```

→ 주석은
코드 실행에
영향 없음

```
for i in range(3):
    print(i)
```

→ 콜론 뒤에
블록 시작

```
if x > 0:
    print(x)
print("done")
```

→ 들여쓰기 종료로
블록 종료

✗ 잘못된 예

```
if x > 0:
    print("양수")
```

→ 콜론(:) 누락

```
Count = 10
print(COUNT)
```

→ 대소문자
혼동으로 오류

```
# 주석입니다
print("Hello")
"""
여러 줄 주석
"
```

→ 문자열 닫는
따옴표 누락

```
for i in range(3)
    print(i)
```

→ 콜론(:) 누락

```
if x > 0:
    print(x)
print("done")
```

→ 들여쓰기가
없어 오류



10 변수와 자료형

Python에서 데이터를 저장하고 다루는 기본 단위



Point

변수와 자료형을 이해하면,
다양한 데이터를 자유롭게 다룰 수 있습니다.



변수란?

데이터를 저장하고, 이름으로 다시 사용할 수 있게 해줍니다.

- ✓ 변수는 데이터를 저장하는 이름입니다
- ✓ Python은 값을 대입하면 타입이 결정됩니다
- ✓ C/C++와 달리 타입 선언을 미리 쓰지 않습니다

> 코드 예시

```
1 temperature = 25
2 name = "sensor_A"
3 is_on = True
```

주요 기본 자료형

123

int

정수

예: 10

3.14

float

실수

예: 3.14

Aa

str

문자열

예: "Hello"



bool

논리값

예: True / False



list

여러 값을 순서대로 저장

예: [23, 25, 24]



핵심 포인트

Python은 변수 선언 시
타입을 따로 쓰지 않는다.

C/C++

```
int a = 10;
```

→ 타입을 먼저 선언해야 함

VS

Python

```
a = 10
```

→ 값이 대입될 때 타입이 결정됨

11 Python 주요 데이터 타입

데이터 타입은 값의 종류와 성질을 정의하며, 적절한 타입을 사용해야 효율적인 프로그래밍이 가능합니다.



Point

Python은 동적 타입 언어로,
변수를 선언할 때 타입을 명시하지 않아도 됩니다.

	타입	설명	예시	코드 예시	결과
123	정수형 (int)	정수를 표현합니다. 양수, 음수, 0을 포함합니다.	10, -5, 0	a = 10 b = -5	a → 10 b → -5
3.14	실수형 (float)	소수점이 있는 숫자를 표현합니다.	3.14, -0.01, 2.0	pi = 3.14 rate = -0.01	pi → 3.14 rate → -0.01
“ ”	문자열 (str)	문자들의 순서(텍스트)를 표현합니다.	"hello", 'Python', "안녕"	msg = "hello" name = 'Python'	msg → "hello" name → 'Python'
True	불리언 (bool)	참(True) 또는 거짓(False)을 나타냅니다.	True, False	is_active = True is_admin = False	is_active → True is_admin → False
[]	리스트 (list)	여러 값을 순서대로 저장하는 자료구조입니다.	[1, 2, 3], ['a', 'b']	nums = [1, 2, 3] fruits = ['a', 'b']	nums → [1, 2, 3] fruits → ['a', 'b']
{ }	딕셔너리 (dict)	키(key)와 값(value)의 쌍으로 저장하는 자료구조입니다.	{'name': '홍길동', 'age': 20}	person = {'name': '홍길동', 'age': 20}	person → {'name': '홍길동', 'age': 20}

TIP type() 함수를 사용하면 변수의 타입을 확인할 수 있습니다.

type(10) → <class 'int'>

type(3.14) → <class 'float'>

type("hi") → <class 'str'>

12 기본 연산자

Python에서 값을 계산하고 비교하고 판단하는 핵심 도구



Point

연산자는 프로그램의 다양한 상황을
정확하고 간단하게 표현해 줍니다.



산술 연산자

+	덧셈
-	뺄셈
*	곱셈
/	나눗셈
//	몫
%	나머지
**	거듭제곱



비교 연산자

==	같다
!=	다르다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다



논리 연산자

and	모두 참
or	하나 이상 참
not	참/거짓 반전

>_ 코드 예시

```

1  x = 10
2  y = 3
3
4  print(x + y)      # 13
5  print(x // y)     # 3
6  print(x % y)      # 1
7  print(x ** 2)     # 100
8
9  print(x > y)       # True
10 print(x == y)      # False
11
12 print(x > 5 and y < 5) # True
13 print(not(x == y))  # True

```



실행 결과

```

13
3
1
100

True
False

True
True

```



핵심 포인트

산술 연산자는 **계산**, 비교 연산자는 **조건 판단**, 논리 연산자는 **여러 조건 결합**에 사용합니다.

연산자는 값을 계산하거나 비교하는 데 사용됩니다.



Point

연산자의 우선순위를 이해하면
복잡한 수식을 정확하게 계산할 수 있습니다.

산술 연산자

연산자	설명	예시	결과
+	덧셈	10 + 3	13
-	뺄셈	10 - 3	7
*	곱셈	10 * 3	30
/	나눗셈 (실수 결과)	10 / 3	3.333...
//	나눗셈 (몫)	10 // 3	3
%	나머지	10 % 3	1
**	거듭제곱	2 ** 3	8

i 산술 연산자 예시

```
a, b = 10, 3
print(a + b) # 13
print(a - b) # 7
print(a * b) # 30
print(a / b) # 3.3333333333333333
print(a // b) # 3
print(a % b) # 1
print(a ** b) # 1000
```

비교 연산자

연산자	설명	예시	결과
==	같다	5 == 5	True
!=	같지 않다	5 != 3	True
>	크다	5 > 3	True
<	작다	5 < 3	False
>=	크거나 같다	5 >= 5	True
<=	작거나 같다	3 <= 5	True

비교 연산자 예시

```
a, b = 5, 3
print(a == b) # False
print(a != b) # True
print(a > b) # True
print(a < b) # False
print(a >= b) # True
print(a <= b) # False
```

논리 연산자

연산자	설명	예시	결과
and	모두 True일 때 True	True and False	False
or	하나라도 True면 True	True or False	True
not	값을 반대로	not True	False

논리 연산자 예시

```
x, y = True, False
print(x and y) # False
print(x or y) # True
print(not x) # False
```



우선순위 (높을수록 먼저 계산)

1. () 2. ** 3. +x, -x, not x 4. *, /, //, % 5. +, - 6. 비교 연산자 7. and 8. or

14 Python 제어문 (조건문과 반복문)

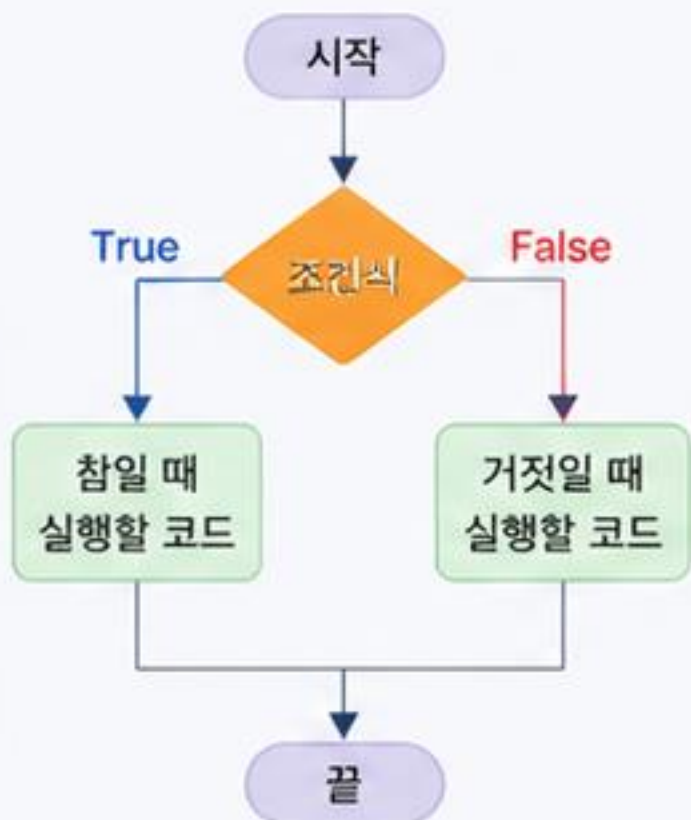
프로그램의 흐름을 제어하여 조건에 따라 다른 동작을 수행하거나, 특정 작업을 반복할 수 있습니다.



Point

제어문을 잘 활용하면 프로그램을 더 유연하고 효율적으로 만들 수 있습니다.

1. 조건문 (if)



```
# 기본 if 문
if 조건식:
    실행할 코드

# if - else 문
if 조건식:
    실행할 코드
else:
    실행할 코드

# if - elif - else 문
if 조건식1:
    실행할 코드
elif 조건식2:
    실행할 코드
else:
    실행할 코드
```

예제 점수에 따른 등급 구하기

```
score = 85
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
else:
    grade = 'D'
print(grade) # B
```

점수 범위	등급
90 이상	A
80 이상 ~ 90 미만	B
70 이상 ~ 80 미만	C
70 미만	D

2. 반복문 (for)

for 변수 in 반복 가능한 객체:
실행할 코드

동작 흐름



예제 리스트의 모든 요소 출력하기

```
fruits = ['사과', '바나나', '포도']
for fruit in fruits:
    print(fruit)
```

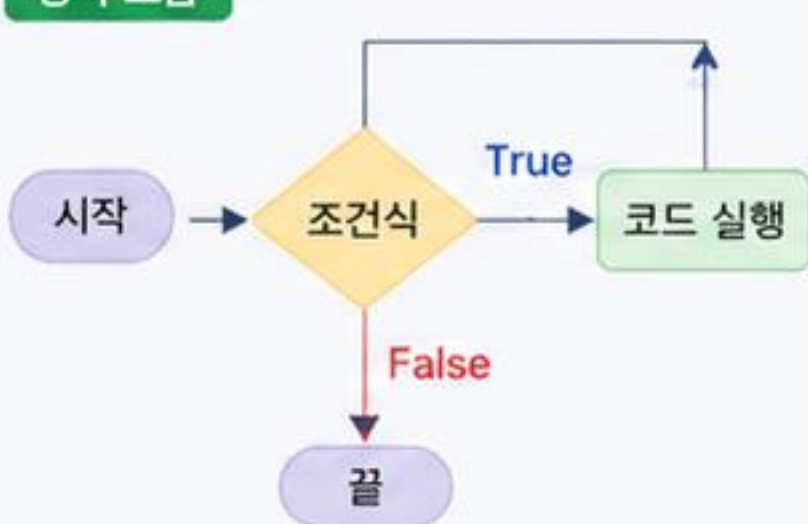
실행 결과

```
사과
바나나
포도
```

3. 반복문 (while)

while 조건식:
실행할 코드

동작 흐름



예제 1부터 5까지 출력하기

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

실행 결과

```
1
2
3
4
5
```

★ 핵심 정리

• 조건문

조건식의 결과(True/False)에 따라 다른 코드를 실행합니다.

• 반복문

특정 작업을 여러 번 반복 수행합니다.

- for 문: 정해진 횟수만큼 반복할 때 사용

- while 문: 조건이 참인 동안 반복할 때 사용

• 들여쓰기(Indentation)

Python은 들여쓰기로 코드 블록을 구분합니다.

들여쓰기를 잘못하면 오류가 발생합니다.

TIP 복잡한 문제도 조건문과 반복문을 조합하면 해결할 수 있습니다!



15 Python 함수 (Function)

함수는 특정 작업을 수행하는 코드를 하나로 묶어 재사용할 수 있게 해줍니다.



Point

함수를 사용하면 코드의 중복을 줄이고, 가독성과 유지보수성을 높일 수 있습니다.

1. 함수 정의

```
def 함수이름(매개변수):  
    수행할 코드  
    return 반환값 # (선택)
```

- def 키워드로 함수를 정의합니다.
- 매개변수(parameter)는 입력값입니다.
- return으로 결과를 반환합니다. (생략 가능)

2. 함수 호출

결과 = 함수이름(인수)
함수이름(인수)

- 함수를 정의한 후에는 호출하여 사용합니다.
- 괄호 안에 인수(argument)를 전달합니다.
- return 값이 있으면 변수에 저장할 수 있습니다.

3. 예제: 두 수의 합을 구하는 함수

```
# 함수 정의  
def add(a, b):  
    result = a + b  
    return result  
  
# 함수 호출  
x = 10  
y = 20  
sum_value = add(x, y)  
print(f"{x} + {y} = {sum_value}")
```

→ add 함수 정의 (매개변수: a, b)
→ 결과 반환
→ 함수 호출 (인수: x, y)

실행 결과

10 + 20 = 30

다양한 호출 예시

```
print(add(5, 7)) # 12  
print(add(-3, 8)) # 5  
  
a, b = 100, 200  
print(add(a, b)) # 300
```

4. 여러 형태의 함수

(1) 반환값이 없는 함수

```
def greet(name):  
    print(f"안녕하세요, {name}님!")  
  
greet("Python") # 안녕하세요, Python님!
```

(2) 매개변수 기본값 사용

```
def greet(name="Guest"):  
    print(f"안녕하세요, {name}님!")  
  
greet() # 안녕하세요, Guest님!  
greet("Alice") # 안녕하세요, Alice님!
```

(3) 여러 개의 반환값

```
def calc(a, b):  
    return a + b, a - b, a * b  
  
s, d, m = calc(10, 3)  
print(s, d, m) # 13 7 30
```

TIP 함수 활용의 장점

✓ 코드 재사용 가능

✓ 코드가 간결해짐

✓ 가독성과 유지보수성 향상

✓ 복잡한 프로그램을 구조화하는 데 도움



17 Python 리스트 (List)

리스트는 여러 개의 값을 순서대로 저장하는 자료구조입니다. 다양한 방법으로 생성, 접근, 수정할 수 있습니다.



Point

리스트는 순서가 있고, 중복을 허용하며, 다양한 자료형을 하나의 리스트에 담을 수 있습니다.

1. 리스트 생성

```
# 빈 리스트 생성
empty = []

# 값이 있는 리스트 생성
numbers = [1, 2, 3, 4, 5]
mixed = [1, "Python", 3.14, True]
```

2. 인덱싱과 슬라이싱

```
fruits = ['사과', '바나나', '포도', '오렌지', '딸기']

# 인덱싱 (0부터 시작)
print(fruits[0])    # 사과
print(fruits[-1])   # 딸기 (뒤에서 첫 번째)

# 슬라이싱
print(fruits[1:4])  # ['바나나', '포도', '오렌지']
print(fruits[:2])   # ['사과', '바나나']
print(fruits[::2])  # ['사과', '포도', '딸기']
```

다양한 리스트 연산 예시

```
a = [1, 2, 3]
b = [4, 5]

print(a + b)  # [1, 2, 3, 4, 5] (연결)
print(a * 2)  # [1, 2, 3, 1, 2, 3] (반복)
```

```
nums = [3, 1, 4, 1, 5]
print(max(nums))  # 5 (최댓값)
print(min(nums))  # 1 (최솟값)
print(sum(nums))  # 14 (합계)
print(nums.count(1))  # 2 (1의 개수)
```

3. 리스트 주요 연산과 함수

연산 / 함수	설명	예시	결과
len(list)	길이	len([1, 2, 3])	3
list.append(x)	끝에 추가	a = [1, 2] a.append(3)	[1, 2, 3]
list.insert(i, x)	i 위치에 삽입	a.insert(1, 10)	[1, 10, 2, 3]
list.remove(x)	x 값 제거(첫 번째)	a.remove(10)	[1, 2, 3]
list.pop(i)	i 위치 값 제거 후 반환 (기본: 마지막)	a.pop() a.pop(0)	3 1
list.clear()	모든 요소 제거	a.clear()	[]
x in list	포함 여부 확인	2 in [1, 2, 3]	True
list.sort()	오름차순 정렬	a.sort()	정렬된 리스트
list.reverse()	순서 뒤집기	a.reverse()	뒤집힌 리스트

4. 리스트 컴프리헨션

```
# 0부터 9까지의 제곱 리스트
squares = [x**2 for x in range(10)]
print(squares)
# 결과: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# 짝수만 추출
evens = [x for x in range(20) if x % 2 == 0]
print(evens)
# 결과: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

5. 중첩 리스트 (2차원 리스트)

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0][1])  # 2 (첫 번째 행의 두 번째 열)
print(matrix[2][2])  # 9 (세 번째 행의 세 번째 열)
```



실행 결과 예시

```
>>> fruits = ['사과', '바나나', '포도']
>>> fruits.append('오렌지')
>>> fruits
['사과', '바나나', '포도', '오렌지']
>>> fruits[1:]
['바나나', '포도', '오렌지']
```

★ 핵심 정리

- 리스트는 순서가 있고 변경 가능한 자료구조입니다.
- 인덱싱과 슬라이싱으로 원하는 요소에 쉽게 접근합니다.
- 다양한 내장 함수로 리스트를 효율적으로 다룰 수 있습니다.
- 리스트 컴프리헨션을 사용하면 간결한 코드를 작성할 수 있습니다.



16 Python 모듈과 패키지 (Module & Package)

모듈은 관련 함수나 변수를 모아둔 파일이고, 패키지는 모듈을 체계적으로 관리하는 폴더 구조입니다.



Point

모듈과 패키지를 잘 활용하면
코드의 재사용성과 유지보수성이 크게 향상됩니다.

1. 모듈 만들기



mymath.py

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
  
PI = 3.14
```

- ✓ mymath.py 파일 생성
- ✓ add, sub 함수 정의
- ✓ PI 변수 정의

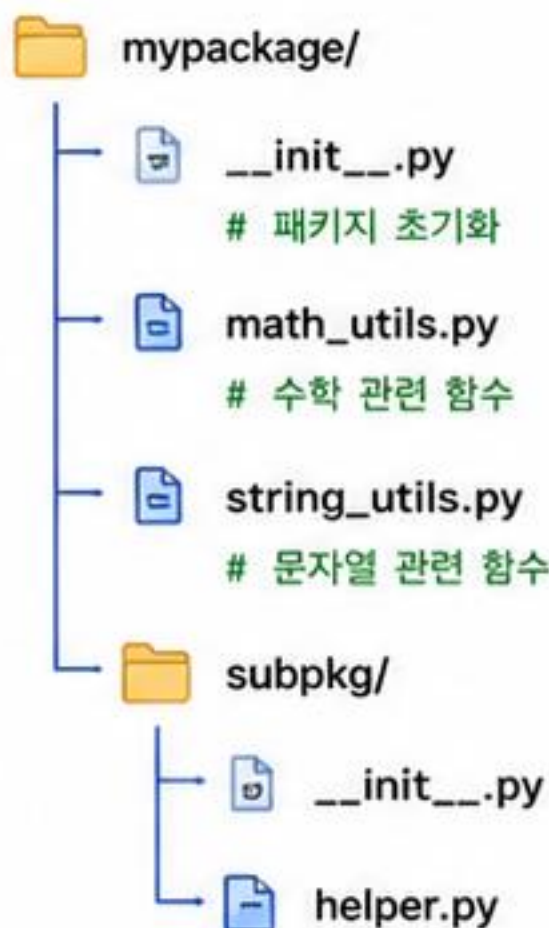
2. 모듈 가져오기

```
import mymath  
  
# 모듈 전체 가져오기  
print(mymath.add(3, 5))  
print(mymath.PI)
```

```
from mymath import add  
  
# 특정 함수만 가져오기  
print(add(10, 20))
```

```
from mymath import *  
  
# 모든 것 가져오기 (권장 X)  
print(sub(10, 3))
```

3. 패키지 구조



4. 패키지 사용 예시

```
from mypackage import math_utils  
result = math_utils.add(7, 8)  
print(result) # 15
```

```
from mypackage.string_utils import  
greet  
print(greet("Python"))  
# 안녕하세요, Python님!
```

```
from mypackage.subpkg import helper  
helper.show_info()
```



핵심 개념

모듈(Module)

- .py 파일 하나
- 함수, 변수, 클래스 포함
- import로 사용

패키지(Package)

- 폴더 구조
- 여러 모듈을 체계적으로 관리
- __init__.py 파일로 패키지 인식

장점

- ✓ 코드 재사용성 향상
- ✓ 네임스페이스 충돌 방지
- ✓ 프로젝트 구조화
- ✓ 유지보수성 증가



실습 예제

간단한 패키지 만들어보기

1

mypackage 폴더
생성

2

__init__.py 파일 생성
(비워둬도 됨)

3

math_utils.py에
계산 함수 작성

4

string_utils.py에
문자열 함수 작성

5

다른 파일에서
패키지 import하여 사용

실행 결과

```
add(10, 5) = 15  
sub(10, 3) = 7  
greet('World') = 안녕하세요, World님!
```



18 Python 딕셔너리 (Dictionary)

딕셔너리는 키(key)와 값(value)의 쌍으로 데이터를 저장하는 자료구조입니다.



Point

딕셔너리는 키를 통해 값을 빠르게 찾을 수 있으며, 해시 기반으로 동작하여 효율적인 검색이 가능합니다.

1. 딕셔너리 생성

```
# 빈 딕셔너리
empty = {}

# 값이 있는 딕셔너리
person = {"name": "Alice", "age": 20, "city": "Seoul"}
print(person) # {'name': 'Alice', 'age': 20, 'city': 'Seoul'}
```

2. 값 접근과 수정

```
person = {"name": "Alice", "age": 20, "city": "Seoul"}

print(person["name"]) # Alice
person["age"] = 21 # 값 수정
print(person["age"]) # 21
person["job"] = "Student" # 새 키-값 추가
print(person)
# {'name': 'Alice', 'age': 21, 'city': 'Seoul', 'job': 'Student'}
```

3. 키 존재 확인과 제거

```
person = {"name": "Alice", "age": 20, "city": "Seoul"}

print("age" in person) # True
print("height" in person) # False

del person["city"] # 키-값 제거
print(person) # {'name': 'Alice', 'age': 20}
```

4. 주요 메서드

메서드	설명	예시
keys()	모든 키를 반환	person.keys()
values()	모든 값을 반환	person.values()
items()	모든 키-값 쌍을 반환	person.items()
get(key)	키에 해당하는 값 반환 (없으면 None)	person.get("age")
pop(key)	키의 값을 반환하고 제거	person.pop("age")
update(other)	다른 딕셔너리의 키-값 쌍을 추가/갱신	person.update(other)
clear()	모든 항목 제거	person.clear()
copy()	얕은 복사본을 반환	person.copy()

예제 학생의 성적 관리

```
scores = {"math": 90, "english": 85, "science": 92}
print(scores.keys()) # dict_keys(['math', 'english', 'science'])
print(scores.values()) # dict_values([90, 85, 92])
print(scores.items()) # dict_items([('math', 90), ('english', 85), ('science', 92)])
print(scores.get("math")) # 90
scores.update({"history": 88}) # 과목 추가
print(scores) # {'math': 90, 'english': 85, 'science': 92, 'history': 88}
removed = scores.pop("english") # 'english' 값 제거
print(removed) # 85
print(scores) # {'math': 90, 'science': 92, 'history': 88}
```

5. 딕셔너리 반복문

```
person = {"name": "Alice", "age": 20, "city": "Seoul"}

# 키로 반복
for key in person:
    print(key) # name, age, city

# 값으로 반복
for value in person.values():
    print(value) # Alice, 20, Seoul

# 키-값 쌍으로 반복
for key, value in person.items():
    print(f"{key}: {value}")
    # name: Alice, age: 20, city: Seoul
```

★ 핵심 정리

- 딕셔너리는 {키: 값} 형태로 데이터를 저장합니다.
- 키는 변경 불가능한 자료형(문자열, 숫자, 튜플 등)을 사용합니다.
- 키를 통해 값을 빠르게 조회할 수 있습니다.
- 다양한 내장 메서드를 제공하여 편리하게 조작할 수 있습니다.

TIP - get(key, default) : 키가 없을 경우 기본값 반환

```
person.get("height", 0) # 0
```

- 딕셔너리는 JSON 데이터와 유사하여 데이터 교환에 많이 사용됩니다.



19 Python 집합 (Set)

집합(Set)은 중복을 허용하지 않는 순서 없는 데이터 모음입니다. 수학의 집합 연산을 지원합니다.



Point

집합은 중복 제거와 빠른 멤버십 검사에 유용하며, 교집합, 합집합, 차집합 등 수학 연산을 지원합니다.

1. 집합 생성

```
# 빈 집합
empty_set = set()

# 값이 있는 집합 (중복 자동 제거)
numbers = {1, 2, 3, 2, 3, 4}      # {1, 2, 3, 4}
fruits = set(['사과', '바나나', '사과']) # {'사과', '바나나'}
```

2. 기본 연산

연산	설명	예시
<code>x in s</code>	원소 x가 집합 s에 포함되어 있는지 확인	<code>2 in {1, 2, 3} → True</code>
<code>x not in s</code>	원소 x가 집합 s에 포함되지 않았는지 확인	<code>4 not in {1, 2, 3} → True</code>
<code>len(s)</code>	집합 s의 원소 개수	<code>len(1, 2, 3) → 3</code>
<code>s.add(x)</code>	원소 x를 집합 s에 추가	<code>s.add(4)</code>
<code>s.remove(x)</code>	원소 x를 집합 s에서 제거 (없으면 오류 발생)	<code>s.remove(2)</code>
<code>s.discard(x)</code>	원소 x를 집합 s에서 제거 (없어도 오류 없음)	<code>s.discard(2)</code>
<code>s.clear()</code>	집합 s의 모든 원소 제거	<code>s.clear()</code>
<code>s.copy()</code>	집합 s의 복사본 반환	<code>s2 = s.copy()</code>

TIP 리스트의 중복 제거: `unique = set(my_list)`
집합을 다시 리스트로: `list(s)` 또는 `sorted(s)`

3. 집합 연산

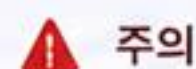
연산	설명	예시
<code>s t</code>	합집합 (union)	<code>{1,2} {2,3} → {1,2,3}</code>
<code>s & t</code>	교집합 (intersection)	<code>{1,2} & {2,3} → {2}</code>
<code>s - t</code>	차집합 (difference)	<code>{1,2,3} - {2} → {1,3}</code>
<code>s ^ t</code>	대칭차집합 (symmetric difference)	<code>{1,2} ^ {2,3} → {1,3}</code>

4. 집합 연산 예제

```
a = {1, 2, 3}
b = {3, 4, 5}
print("a =", a)      # a = {1, 2, 3}
print("b =", b)      # b = {3, 4, 5}

print("합집합:", a | b) # {1, 2, 3, 4, 5}
print("교집합:", a & b) # {3}
print("차집합 (a-b):", a - b) # {1, 2}
print("대칭차집합:", a ^ b) # {1, 2, 4, 5}

a.add(6)
a.remove(2)
print("수정된 a:", a) # {1, 3, 6}
```



주의

()는 딕셔너리, set()은 빈 집합을 생성합니다.
빈 집합: `s = set()` (X) `s = {}` → 딕셔너리

실행 결과 예시

```
a = {1, 2, 3}
b = {3, 4, 5}
합집합: {1, 2, 3, 4, 5}
차집합 (a-b): {1, 2}
수정된 a: {1, 3, 6}
```

```
교집합: {3}
대칭차집합: {1, 2, 4, 5}
```

6. 불변성과 가변성

- 집합은 가변(mutable) 자료형입니다.
- 집합의 원소는 변경할 수 없어야 합니다. (리스트, 딕셔너리, 집합은 원소로 사용할 수 없음)
- 집합은 해시 기반으로 동작하여 멤버십 검사가 빠릅니다.

★ 핵심 정리

- 집합은 중복을 허용하지 않고 순서가 없습니다.
- 수학의 집합 연산(합집합, 교집합, 차집합 등)을 지원합니다.
- 멤버십 검사(in 연산)가 매우 빠릅니다.
- set()로 생성하며, 중괄호 {}를 사용합니다.
- 집합 컴프리헨션으로 간결하게 생성할 수 있습니다.

20 Python 튜플 (Tuple)

튜플(Tuple)은 변경할 수 없는 순서가 있는 데이터 모음입니다. 주로 서로 연관된 데이터를 안전하게 저장할 때 사용합니다.

 **Point**

튜플은 생성 후 값을 변경, 추가, 삭제할 수 없어서 데이터의 무결성을 보장할 때 유용합니다.

1. 튜플 생성

```
# 기본 튜플 생성 (소괄호 생략 가능)
t1 = (1, 2, 3)
# 소괄호 생략
t2 = 4, 5, 6
# 한 개의 요소
t3 = (7,)      # 콤마 필수!
# 빈 튜플
t4 = tuple()
print(t1, t2, t3, t4)
# (1, 2, 3) (4, 5, 6) (7,) ()
```

2. 인덱싱과 슬라이싱 (읽기 전용)

```
t = ('apple', 'banana', 'cherry', 'date')
print(t[0])      # apple
print(t[-1])     # date
print(t[1:3])    # ('banana', 'cherry')
```

3. 주요 메서드

메서드	설명	예시
count(x)	x의 개수 반환	t.count('banana')
index(x)	x의 첫 번째 인덱스 반환	t.index('cherry')
len(t)	튜플의 길이 반환	len(t)

★ **핵심 정리**

• 튜플은 순서가 있고, 변경할 수 없는(immutable) 자료형입니다.

• 인덱싱, 슬라이싱, 연산은 가능하지만 값 수정은 불가능합니다.

• 작은 단위의 고정된 데이터를 저장하고, 반환할 때 유용합니다.

4. 튜플 연산

연산	설명	예시	결과
t1 + t2	튜플 결합 (concatenation)	(1,2) + (3,4)	(1, 2, 3, 4)
t * n	튜플 반복	('a',) * 3	('a', 'a', 'a')
x in t	포함 여부 확인	2 in (1,2,3)	True
x not in t	미포함 여부 확인	5 not in (1,2,3)	True

5. 튜플 언패킹

```
point = (10, 20)
x, y = point      # 언패킹
print(x, y)       # 10 20

a, b, *rest = (1, 2, 3, 4, 5)
print(a, b, rest) # 1 2 [3, 4, 5]

first, *middle, last = ('a', 'b', 'c', 'd')
print(first, middle, last) # a ['b', 'c'] d
```

TIP 여러 값을 반환할 때 유용!

```
def min_max(data):
    return min(data), max(data)

print(min_max([3, 1, 4, 1, 5])) # (1, 5)
```

6. 튜플은 변경 불가 (Immutable)

```
t = (1, 2, 3)
# t[0] = 10      # TypeError 발생
# t.append(4)     # AttributeError 발생
# t.remove(2)     # AttributeError 발생

# 새로운 튜플 생성은 가능
t2 = t + (4,)
print(t2)        # (1, 2, 3, 4)
```

7. 튜플 활용 예시

```
# 좌표 저장
point = (37.5665, 126.9780)
# 사용자 정보 (변경되지 않아야 할 데이터)
user = ('user01', '홍길동', 'user01@example.com')
# 여러 값 반환
def divide(a, b):
    return a // b, a % b    # 몫과 나머지 튜플로 반환

q, r = divide(17, 5)
print(q, r)               # 3 2
```

실행 결과 예시

(1, 2, 3) (4, 5, 6) (7,) ()

10 20

apple

1 2 [3, 4, 5]

date

a ['b', 'c'] d

('banana', 'cherry')

(1, 2, 3, 4)

10 20

3 2

1 2 [3, 4, 5]